计算机网络实验 2
编写数据包捕获程序并对捕获的数据包进行分析

鲁昕宁 2414015

2026 年 11 月 10 日

1 实验目的

用 C++ 和 Npcap 在本机捕获以太网帧，并把解析结果直接显示在窗口里。实验要完成四件事：

1. 了解 Npcap 从用户态接口到内核驱动的结构；
2. 用 Npcap 枚举网卡、打开设备并读取数据包；
3. 从每一帧中取出源 MAC 地址、目的 MAC 地址、类型字段，以及类型字段之后的第一个字节；
4. 不把数据包存到硬盘，只在屏幕上按列显示这些字段。

2 实验环境

程序使用 C++17 和 WinUI 3。捕获接口来自 Npcap SDK 1.17 的 `pcap.h`，运行时链接本机的 Npcap 1.90。用户态库 `wpcap.dll` 基于 libpcap 1.10.7，驱动文件是 `npcap.sys`。安装时打开了 WinPcap API 兼容模式，因此程序按 libpcap 的方式调用即可。注册表中的 `AdminOnly` 为 0，当前用户可以直接打开网卡。

编译环境为 Visual Studio 2022 Professional，平台工具集 v143，Windows SDK `10.0.22621.0`，Windows App SDK 1.6。工程带自包含运行库。可执行文件是 `capture.exe`，位于 `bin/capture`。

3 Npcap 架构

Npcap 把抓包分成四层。程序只接触最上面的一层。

表 1: Npcap 的层次

层次	模块	本实验用到的动作
用户程序	`capture.exe`	选择网卡、解析以太网头、刷新列表
libpcap 兼容接口	`wpcap.dll`	`pcap_findalldevs`、`pcap_open_live`、`pcap_next_ex`
底层接口	`Packet.dll`	把上面的调用交给内核驱动
内核驱动	`npcap.sys`	作为 NDIS 轻量过滤器挂在网卡上，把帧拷进缓冲

`pcap_findalldevs` 返回驱动能看到的适配器。名字类似 `\Device\NPF_{GUID}`，描述是网卡在系统里的名称。`pcap_open_live` 按这个名字打开一台适配器，并指定每次最多拷贝的字节数、是否混杂模式，以及读操作的超时。打开之后，驱动把经过该网卡的帧放进内核缓冲，`pcap_next_ex` 从缓冲里取出下一帧。超时返回 0，调用 `pcap_breakloop` 返回 -2。

本程序在打开后检查 `pcap_datalink`。只有 `DLT_EN10MB` 才按以太网头解析。回环等设备没有这 14 字节头，会在状态行说明并停止，避免把别的链路头误当成 MAC 地址。

4 以太网帧与程序结构

4.1 帧布局

四个必显字段在以太网帧头中的位置是固定的。前 12 字节是目的 MAC 和源 MAC，各 6 字节。偏移 12 的两个字节是类型字段，按网络字节序合成一个 16 位整数。偏移 14 就是类型字段后

第一个字节。

类型字段与协议的对应关系是：0x0800 表示 IPv4，0x0806 表示 ARP，0x86DD 表示 IPv6，0x8100 表示 VLAN。类型字段为 0x8100 时，这一帧带有 VLAN 标记，类型字段后第一个字节是标记控制信息的高字节，内层类型字段还在其后。程序仍按偏移 12 和偏移 14 读取。捕获长度不足 15 字节时，类型字段和类型字段后第一个字节无法取全。这样的帧不进入列表，状态行显示已经跳过多少帧。

```
EthernetView parse_ethernet(const uint8_t* data, std::size_t length)
{
    EthernetView view;
    if (data == nullptr || length < 15)
    {
        return view;
    }
    for (int i = 0; i < 6; ++i)
    {
        view.dst[i] = data[i];
        view.src[i] = data[6 + i];
    }
    view.type = static_cast<uint16_t>((static_cast<uint16_t>(data[12]) << 8) |
    ↪  data[13]);
    view.after_type = data[14];
    view.ok = true;
    return view;
```

类型为 0x0800 时，类型字段后第一个字节是 IPv4 首部的第一个字节。它的高 4 位是版本号，低 4 位是首部长度，单位为 4 字节。首部不含可选项时，这一字节是 0x45，也就是版本 4、首部长 20 字节。这一字节是 0x46 时，版本仍是 4，首部长度字段是 6，首部长 24 字节，其中 4 字节是可选项。类型为 0x86DD 时，类型字段后第一个字节是 IPv6 首部的第一个字节，高 4 位是版本号。0x60 表示版本 6，低 4 位是通信量类的高半字节，这里是 0。类型为 0x0806 时，类型字段后第一个字节是 ARP 硬件类型的高字节。以太网的硬件类型是 1，所以这个字节通常是 0x00。

4.2 源码目录

界面和捕获分开。窗口工程只负责下拉框、按钮和列表，Npcap 调用集中在 core。

```
lab2/
  Capture.sln            解决方案
  build.bat              调用 MSBuild 生成 Release x64
  core/
    ethernet.h/.cpp      按偏移解析 MAC、类型和其后第一个字节
    capture.h/.cpp       设备列表、打开网卡、捕获线程
  app/                   WinUI 窗口
    MainWindow.xaml.cpp  网卡、条数、开始、停止和结果列表
  third_party/npcap/     Npcap SDK 1.17
  bin/capture/           可执行文件和自包含运行库
```

设备列表在窗口启动时填充。描述字符串经 pcap_init(PCAP_CHAR_ENC_UTF_8) 后按 UTF-8 转成宽字符。

```
    if (pcap_findalldevs(&all, errbuf) != 0)
    {
        error = to_wide(errbuf);
        if (error.empty())
        {
            error = L"pcap_findalldevs 失败";
        }
        return devices;
    }
    for (pcap_if_t* it = all; it != nullptr; it = it->next)
    {
        Device device;
        device.name = it->name != nullptr ? it->name : "";
        const char* text = (it->description != nullptr && it->description[0] !=
        ↪  '\0') ? it->description : it->name;
        device.label = to_wide(text);
        if (device.label.empty())
        {
            device.label = L"（未命名设备）";
        }
        devices.push_back(std::move(device));
    }
    pcap_freealldevs(all);
```

打开网卡时快照长度为 65535，混杂模式打开，读超时 1000 毫秒。超时让捕获线程能在点击停止后从 pcap_next_ex 返回。

```
    char errbuf[PCAP_ERRBUF_SIZE]{};
    // 65535 字节足够装下一帧以太网。混杂模式打开，超时 1000 毫秒以便停止时能返回。
    pcap_t* handle = pcap_open_live(device_name.c_str(), 65535, 1, 1000, errbuf);
    if (handle == nullptr)
    {
        error = to_wide(errbuf);
        if (error.empty())
        {
            error = L"pcap_open_live 失败";
        }
        return false;
    }
    if (pcap_datalink(handle) != DLT_EN10MB)
    {
        pcap_close(handle);
        error = L" 所选设备不是以太网，不能按 MAC 帧解析";
        return false;
    }
```

捕获循环在单独的线程里运行。条数为 0 时一直读，直到 pcap_breakloop。每得到一帧就填好一行，通过回调交给窗口。

```
        while (!impl_->stopping.load())
        {
```

```
156             if (limit != 0 && captured >= limit)
157             {
158                 break;
159             }
160             pcap_pkthdr* header = nullptr;
161             const u_char* data = nullptr;
162             const int rc = pcap_next_ex(handle, &header, &data);
163             if (rc == 0)
164             {
165                 continue;
166             }
167             if (rc == -2)
168             {
169                 break;
170             }
171             if (rc < 0)
172             {
173                 failure = to_wide(pcap_geterr(handle));
174                 if (failure.empty())
175                 {
176                     failure = L"pcap_next_ex 失败";
177                 }
178                 break;
179             }
180             if (header == nullptr || data == nullptr || header->caplen < 15)
181             {
182                 ++skipped;
183                 if (on_status)
184                 {
185                     on_status(L"正在捕获，已跳过 " + std::to_wstring(skipped) +
                        ↪  L" 个不足 15 字节的帧");
186                 }
187                 continue;
188             }
189             const EthernetView view = parse_ethernet(data, header->caplen);
190             if (!view.ok)
191             {
192                 continue;
193             }
194             FrameLine line;
195             line.index = ++captured;
196             line.length = header->len;
197             line.dst = format_mac(view.dst);
198             line.src = format_mac(view.src);
199             line.type = format_type(view.type);
200             line.next = format_byte(view.after_type);
201             if (on_frame)
202             {
203                 on_frame(line);
```

```
        }
```

4.3 界面线程

捕获线程不接触控件。窗口用 `post` 把追加一行和改状态行排到 UI 线程，和实验 1 的做法相同。窗口关闭时先把 `open_` 置假，再停止捕获线程，避免关闭过程中再改已经销毁的控件。

```
void MainWindow::post(std::function<void()> action)
{
    if (!open_.load())
    {
        return;
    }
    DispatcherQueue().TryEnqueue([this, action = std::move(action)] {
        if (!open_.load())
        {
            return;
        }
        action();
    });
}
```

工具栏有网卡下拉框、刷新、条数、开始捕获和停止。下方列表的列是序号、长度、目的 MAC、源 MAC、类型和类型后首字节。列表最多保留最近 1000 行，避免长时间捕获时窗口越来越慢。

5 运行结果

在 Intel Wi-Fi 6 AX200 160MHz 上把条数填为 0，捕获一直进行到点击停止。状态行显示“捕获结束，共 6158 帧”。列表只保留最近 1000 行，所以画面里最早的序号是 5159。窗口见图 1。

画面里能看到两类单播地址。本机这块无线网卡的 MAC 是 `74:D8:3E:04:17:7B`。网关地址是 `00:00:5E:00:01:FE`，这是 IANA 为 VRRP 保留的虚拟路由器 MAC。从本机发出的帧，源地址是网卡 MAC、目的地址是网关；网关发回的帧方向相反。序号 5160 就是网关发来的一帧，长度 1514，类型 `0x0800`，类型后第一个字节 `0x45`。1514 字节等于 14 字节以太网头加上 1500 字节负载，是去掉帧校验后的以太网最大长度。同一段列表里还有 54、66、78、86、90、109 和 282 字节的帧，每一行保留的是该帧自己的捕获长度。

类型字段与协议的对应关系在这批帧里都能对上。`0x0800` 的帧，类型后第一个字节多数是 `0x45`，版本 4、首部长 20 字节。序号 5167 这一字节是 `0x46`，版本仍是 4，首部长度字段是 6，首部长 24 字节，多出的 4 字节是可选项。`0x86DD` 的帧，类型后第一个字节都是 `0x60`，版本 6，通信量类的高半字节为 0。

序号 5167 和 5168 的源地址都是 `BA:43:C1:4C:A4:A3`。这个地址的首字节是 `0xBA`，本地管理位为 1。5167 的目的地址是 `01:00:5E:00:00:16`。IPv4 组播的以太网目的地址由 `01:00:5E` 和组播地址的低 23 位组成。`224.0.0.22` 的低 23 位是 `00:00:16`，IGMP 把报文发往这个地址。5168 的目的地址是 `33:33:00:00:00:16`。IPv6 组播的以太网目的地址由 `33:33` 和组播地址的最后 32 位组成。`ff02::16` 的最后 32 位是 `00:00:00:16`，MLD 把报文发往这个地址。这两帧一条是 IPv4、一条是 IPv6，类型后第一个字节分别是 `0x46` 和 `0x60`。

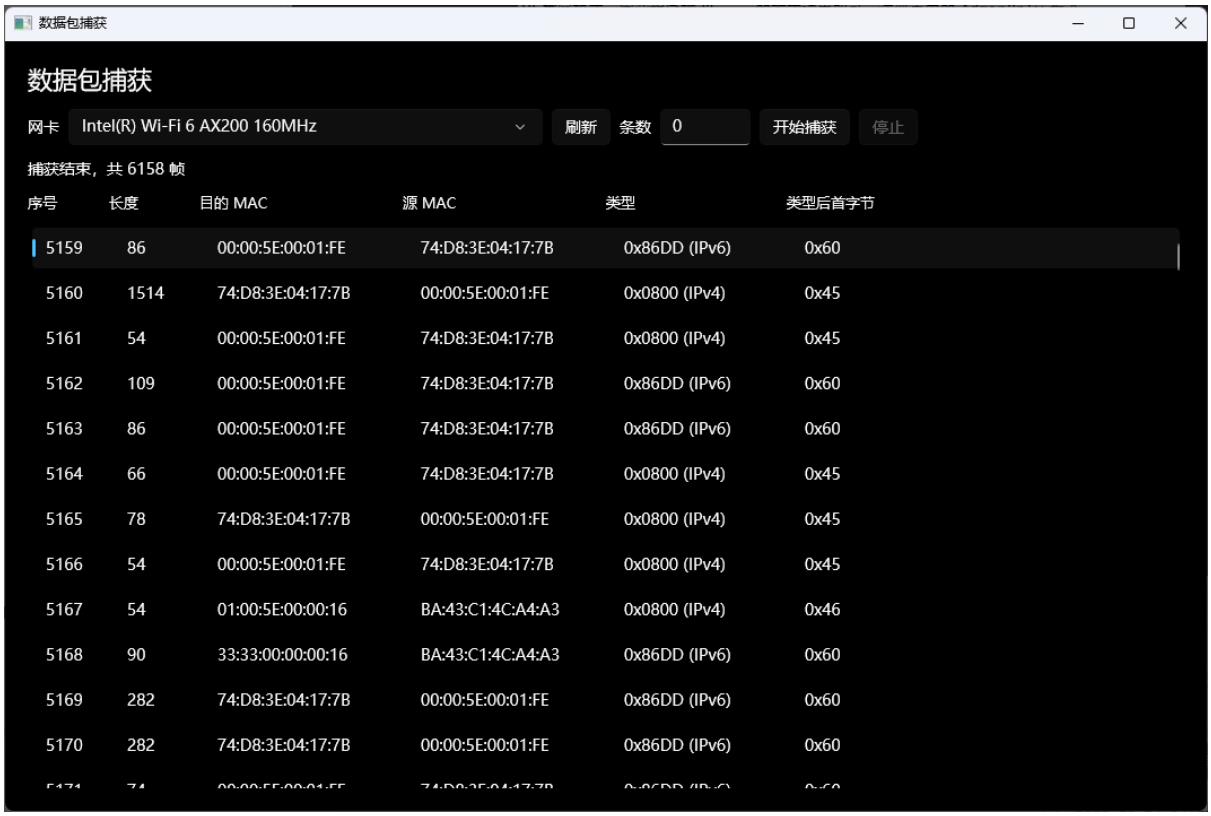


图 1: 在无线网卡上持续捕获到停止，共 6158 帧。画面中同时有 IPv4 和 IPv6。

表 2: 画面中的四帧

序号	长度	目的 MAC	源 MAC	类型	类型后首字节
5159	86	00:00:5E:00:01:FE	74:D8:3E:04:17:7B	0x86DD	0x60
5160	1514	74:D8:3E:04:17:7B	00:00:5E:00:01:FE	0x0800	0x45
5167	54	01:00:5E:00:00:16	BA:43:C1:4C:A4:A3	0x0800	0x46
5168	90	33:33:00:00:00:16	BA:43:C1:4C:A4:A3	0x86DD	0x60

6 结论

程序通过 Npcap 的 `wpcap.dll` 枚举网卡、打开以太网设备，并在独立线程里读取帧。窗口按固定偏移显示目的 MAC、源 MAC、类型字段和类型字段之后的第一个字节，不把数据包写到硬盘。在 Intel Wi-Fi 6 AX200 上持续捕获到停止时，状态行显示共 6158 帧。画面中有本机与网关之间的单播，也有 IPv4 和 IPv6 组播。类型 `0x0800` 后第一个字节是 `0x45` 或 `0x46`，类型 `0x86DD` 后第一个字节是 `0x60`。